

Manual de Ejecución

Proyecto Big Data — Análisis de Reseñas de Hoteles

Curso: Big Data - Grupo 33 - B III 2026

Profesor: Gerardo Nereo Campos Araya

Estudiantes

Andrés Esquivel Valerio 402500696

María Fernanda Granados Arias 402200492

Liz Madriz Rivera 117860659

Estructura del proyecto

```
Big_Data_Proyecto/
├── README.md
├── data/                raw (CSVs) + processed (parquet); no versionados
├── docker/             Dockerfile, docker-compose.yml, requirements.txt
├── docs/               ManualEjecucion, GUIA_EJECUCION, embeddings_teoría,
                        kibana_dashboard.ndjson, diagrams/ (PNGs)
├── src/
│   ├── etl/            loader, cleaner, exporter, embeddings, run_etl
│   ├── db/             es_setup, es_indexer, postgres_writer,
│                       main_materializacion, schemas/postgres_schema.sql
│   ├── search/         generate_query (búsqueda semántica)
│   └── models/         pipeline, logistic_regression, random_forest,
│                       main_models, resultados.ipynb
└── tests/             conftest, test_loader, test_cleaner,
                        test_exporter, test_embeddings
```

Instrucciones de ejecución

Requisito previo: Docker Desktop en ejecución (8–12 GB de RAM) y los archivos Datafiniti_Hotel_Reviews.csv y states.csv colocados en data/raw/ (no van en el repo).

1. Levantar el entorno

Desde la carpeta docker/:

```
cd docker
docker compose build spark      # 1ra vez tarda: baja torch + el modelo
docker compose up -d           # Spark, Postgres, Elasticsearch, Kibana
docker ps
```

Interfaces: Spark UI <http://localhost:8080> · Elasticsearch <http://localhost:9200> · Kibana <http://localhost:5601>

2. Entrar al contenedor de Spark

```
docker exec -it spark bash
cd /opt/spark
export PYTHONPATH=/opt/spark/src:/opt/spark/src/etl
```

3. ETL (carga, limpieza, cruce y embeddings)

Programa principal del ETL. Sin la bandera corre rápido (sin embeddings); los embeddings son un paso pesado de una sola vez:

```
/opt/spark/bin/spark-submit src/etl/run_etl.py          # carga +
limpieza + parquet limpio
/opt/spark/bin/spark-submit src/etl/run_etl.py --with-embeddings # además genera
los embeddings
```

4. Pruebas unitarias

pyspark vive en \$SPARK_HOME/python, hay que agregarlo al path:

```
export PYTHONPATH=/opt/spark/src:/opt/spark/python:$(ls
/opt/spark/python/lib/py4j-*-src.zip)
python3 -m pytest tests/ -v          # 15 passed
```

5. Materialización en PostgreSQL y Elasticsearch

Programa principal que escribe en ambas bases de un solo comando (crea los índices de ES, indexa los documentos y escribe las tablas de Postgres). El driver JDBC de Postgres y el cliente de Elasticsearch ya vienen en la imagen:

```
export PYTHONPATH=/opt/spark
/opt/spark/bin/spark-submit src/db/main_materializacion.py
```

Validación con SQL:

```
docker exec -it postgres psql -U bigdata -d hotel_reviews -c \
"SELECT region, COUNT(*), ROUND(AVG(rating),2) FROM hotel_reviews GROUP BY
region;"
```

Validación de los índices de Elasticsearch:

```
curl -s "localhost:9200/_cat/indices/hotel-reviews-*?v&h=index,docs.count"
```

6. Búsqueda semántica

El script genera dos consultas (BM25 sin vectores y kNN con vectores) a partir de una pregunta en lenguaje natural, para comparar cuál funciona mejor. Pasos:

```
docker exec -it spark /bin/bash
cd /opt/spark
python3 src/search/generate_query.py
```

- Escribir la búsqueda de prueba cuando el script lo solicite.
- Copiar el JSON de BM25 y el JSON de kNN que arroja el script.
- Pegarlos en Kibana > Dev Tools y presionar el botón Play.
- Comparar los resultados con vectores (kNN) vs. sin vectores (BM25).

7. Modelos de predicción

Entrena Logistic Regression y Random Forest y compara métricas:

```
export PYTHONPATH=/opt/spark
/opt/spark/bin/spark-submit src/models/main_models.py
```

8. Dashboards de Kibana

Dashboard con 5 visualizaciones. Para visualizarlo:

- Ingresar a Kibana (<http://localhost:5601>).
- Dirigirse a Stack Management > Saved Objects.
- Dar clic en el botón Import (esquina superior derecha).
- Seleccionar el archivo docs/kibana_dashboard.ndjson incluido en el repositorio.
- Dirigirse a la sección Dashboards para ver las visualizaciones generadas.

9. Detener el entorno

```
exit
docker compose down
```

Resultados

- ETL: 25.600 reseñas cargadas, limpiadas (filtro de idioma inglés + rating 1–5) y cruzadas con estados (region/division).
- Embeddings: vectores de 384 dimensiones por reseña (all-MiniLM-L6-v2).
- Pruebas unitarias: 15 pruebas en verde (loader, cleaner, exporter, embeddings).
- PostgreSQL: tabla hotel_reviews con 25.600 filas (más las fuentes crudas hotels_raw y states, antes del cruce).
- Elasticsearch: dos índices, hotel-reviews-bm25 (sin vectores) y hotel-reviews-semantic (con vectores, 384 dims), con 25.506 documentos cada uno.
- Modelos: comparación Logistic Regression vs Random Forest (Accuracy, F1, AUC-ROC); análisis en src/models/resultados.ipynb.
- Búsqueda semántica: comparación de resultados con vectores (kNN) vs sin vectores (BM25) en Kibana.
- Dashboards: 5 visualizaciones en Kibana sobre los datos almacenados.